



Data Structure Programming Report

Assignment 03

Professor	Sangho Choi
Department	Computer engineering
Student ID	2020202090
Name	Minseok Choi
Submission Date	2023. 12. 14

1. Introduction

이번 데이터구조설계및실습 3 차 프로젝트는 그래프를 이용해 그래프 연산을 수행할 수 있는 프로그램을 구현하는 것이다.

그래프의 정보는 텍스트 파일을 통해 프로그램에 입력하고, 그래프의 특성에 따라 너비 우선 탐색(BFS), 깊이 우선 탐색(DFS), Kruskal 알고리즘, Dijkstra 알고리즘, Bellman-Ford 알고리즘, FLOYD 알고리즘, 그리고 과제 파일로 조건이 주어지는 KwangWoon 알고리즘까지 총 7 가지의 그래프 연산 알고리즘을 수행한다.

KwangWoon 알고리즘은 항상 그래프의 정점 1 부터 탐색을 시작하여 현재 연결되어 있는 엣지의 개수가 짝수이면 가장 작은 정점 번호로 방문하고, 홀수라면 가장 큰 정점의 번호로 방문한다. 세그먼트 트리는 현재 정점에서 다른 정점으로 이동할 수 있는 이동 여부를 위해 사용된다. 따라서 리프 노드는 다른 정점으로의 이동 여부를 담고 있다. 만약 그래프에 1 번 정점이 없다면 에러를 출력한다.

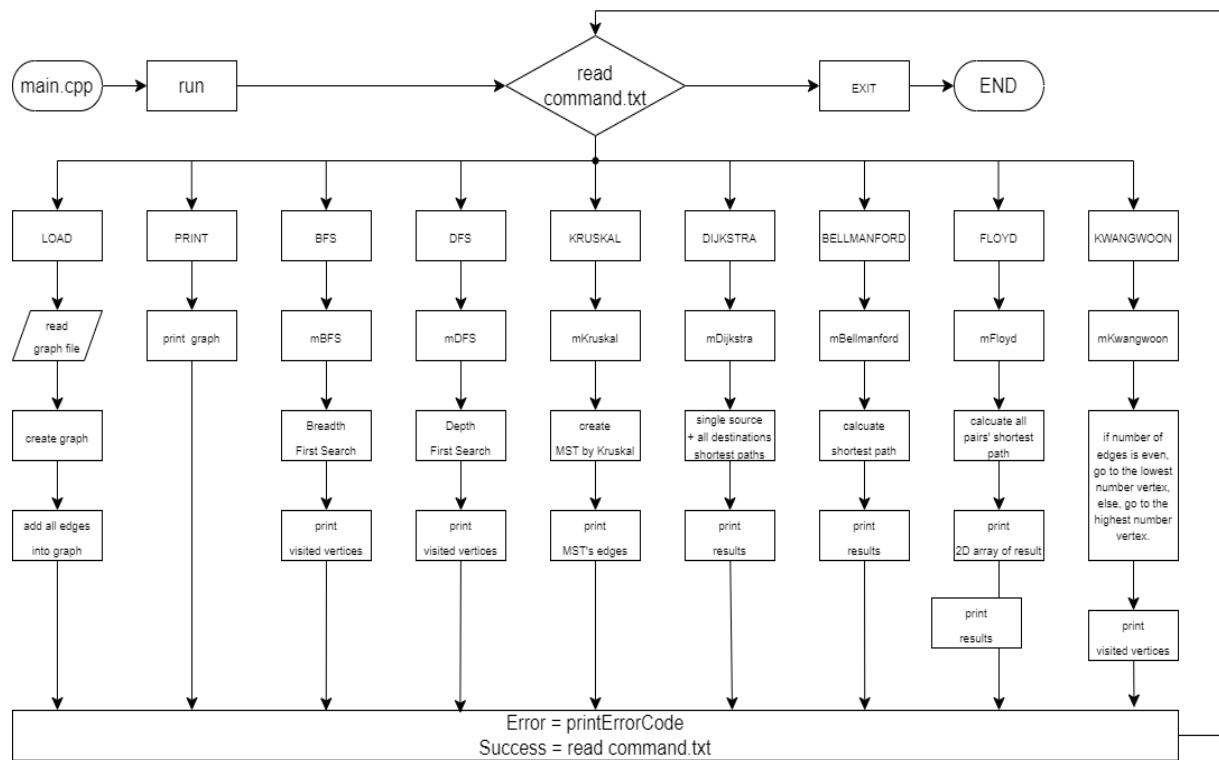
주어지는 그래프 데이터는 기본적으로 방향성 (direction)과 가중치 (weight)를 모두 가지고 있으며 그래프 정보의 형태에 따라 인접 리스트를 사용하는 리스트 그래프와 인접 행렬을 사용하는 행렬 그래프로 저장한다.

위 모든 과정을 Manager 를 통해 관리하며 command.txt 에 입력된 명령어를 통해 프로그램을 제어한다. 제어 중 오류 발생 시 log.txt 에 알맞은 오류 코드를 출력하며, 프로그램 수행 결과는 모두 log.txt 에 출력한다.

다음은 명령어에 대한 구체적인 조건이다.

명령어	기능
LOAD	텍스트 파일로부터 그래프의 정보를 프로그램에 불러온다. 새로운 그래프를 LOAD 하는 경우 기존 그래프는 삭제한다. 오류 발생시 오류 코드 100 번을 출력한다.
PRINT	그래프의 유형에 따라 인접 행렬 혹은 인접 리스트를 출력한다. vertex 는 오름차순으로 출력한다. 오류 발생시 오류 코드 200 번을 출력한다.
BFS	Y 가 입력된 경우 방향성 그래프, N 이 입력된 경우 무방향성 그래프로 간주하고 시작 정점을 기준으로 BFS 를 수행한다. 방문한 정점을 순서대로 출력한다. 오류 발생시 오류 코드 300 번을 출력한다.
DFS	Y 가 입력된 경우 방향성 그래프, N 이 입력된 경우 무방향성 그래프로 간주하고 시작 정점을 기준으로 DFS 를 수행한다. 방문한 정점을 순서대로 출력한다. 오류 발생시 오류 코드 400 번을 출력한다.
KRUSKAL	현재 그래프의 MST 를 구하고 이를 구성하는 간선들의 가중치 값의 총합을 출력한다. 오류 발생시 오류 코드 600 번을 출력한다.
DIJKSTRA	그래프의 방향성을 고려하여 시작 정점을 기준으로 최단 경로와 코스트를 출력한다. 도달할 수 없는 경우 x 를 출력한다. 오류 발생시 오류 코드 700 번을 출력한다.
BELLMANFORD	그래프의 방향성을 고려하여 시작 정점을 기준으로 최단 경로와 코스트를 출력한다. 음수 가중치가 존재하는 그래프에서도 동작해야 한다. 도달할 수 없는 경우 x 를 출력한다. 오류 발생시 오류 코드 800 번을 출력한다.
FLOYD	그래프의 방향성을 고려하여 모든 정점 쌍에 대하여 시작 정점에서 도착 정점으로 가는 데 필요한 비용의 최솟값을 행렬 형태로 출력한다. 도달할 수 없는 경우 x 를 출력한다. 음수 사이클이 발생한 경우 오류를 출력한다. 오류 발생시 오류 코드 900 번을 출력한다.
KWANGWOON	KwangWoon 알고리즘에 따라 방문하는 정점을 출력한다. 리스트 그래프만을 사용한다. 오류 발생시 오류 코드 500 번을 출력한다.
EXIT	프로그램을 종료하고 모든 메모리를 해제한다.

2. Flowchart



3. Algorithm

- Graph

그래프 정보를 담기 위해 사용하는 ListGraph와 MatrixGraph는 모두 이 클래스를 상속받아 사용한다. 멤버 변수로 m_Type과 m_Size를 가지며 m_Type이 true면 MatrixGraph, false면 ListGraph로 구분한다. 생성자에선 두 멤버변수를 입력 받아 초기화하며 이를 반환하는 함수로 구성된다.

- ListGraph

그래프의 엣지 정보를 담기 위해 인접 리스트를 사용하는 형태의 그래프 클래스이다. 입력 받은 사이즈에 따라 엣지를 담는 map을 동적 할당하며 KwangWoon 알고리즘에 사용하기 위한 벡터 리스트도 동적 할당하여 초기화해 준다. 멤버 함수는 그래프에서 어떤 정점의 엣지를 방향성으로 반환하는 함수, 무방향성으로 반환하는 함수, 인접 리스트 정보를 출력하는 함수로 구성된다.

- MatrixGraph

그래프의 엣지 정보를 담기 위해 인접 행렬을 사용하는 형태의 그래프 클래스이다. 입력 받은 사이즈에 따라 엣지를 담는 2차원 정수형 배열을 동적 할당하며 초기화도 진행한다. 멤버 함수는 리스트 그래프와 동일하게 어떤 정점의 엣지를 방향성, 무방향성으로 반환하는 함수와 인접 행렬의 정보를 출력하는 함수로 구성된다.

- GraphMethod

그래프에 대한 연산을 수행하는 함수의 집합이다.

- bool BFS(Graph* graph, char option, int vertex)

그래프에 대한 너비 우선 탐색을 수행한다. 먼저 각 정점별로 방문 여부를 확인하는 배열을 선언하고 탐색 지점을 큐를 이용해서 우선순위를 정한다.

시작 지점을 큐에 삽입한 후 큐가 빌 때까지 탐색을 실시한다. 그래프의 getAdjacentEdges 함수를 이용해 현재 정점에서 이동 가능한 지점을 map 형태로 받아오며 각 정점에 대해 방문 여부를 체크하고 방문하지 않았다면 다음 탐색 지점을 큐에 삽입한다.

- bool DFS(Graph* graph, char option, int vertex)

그래프에 대한 깊이 우선 탐색을 수행한다. BFS와 비슷하게 각 정점 별로 방문 여부를 확인하는 배열을 선언하고 탐색 지점을 스택을 이용해서 우선순위를 정한다.

시작 지점을 스택에 삽입한 후 스택이 빌 때까지 탐색을 실시한다. 그래프의 getAdjacentEdges 함수를 이용해 현재 정점에서 이동 가능한 정점을 map 형태로 받아오며 각 정점에 대한 방문 여부를 체크하고 방문하지 않았다면 다음 탐색 지점을 스택에 삽입한다.

- bool KWANGWOON(Graph* graph, int vertex)

그래프의 1번 정점에서 시작하며 해당 정점과 연결된 정점의 개수가 짝수면 가장 작은 번호의 정점으로, 홀수면 가장 큰 번호의 정점으로 탐색을 실시한다. BFS, DFS와 유사하게 방문 여부를 확인하는 배열을 선언하고 탐색 지점은 큐를 이용해서 우선순위를 관리한다.

전반적으로 BFS와 유사한 형태로 동작하지만 여러 개의 지점을 동시에 큐에 삽입하진 않고 한 번에 한 지점만 큐에 삽입한다.

- `bool Kruskal(Graph* graph)`

그래프에 대한 크루스칼 MST 생성 알고리즘을 수행하는 함수이다. 먼저 크루스칼 알고리즘에서 사이클을 검출하기 위해 서로소 집합 구조체와 유니온, 파인드 함수를 직접 구현한다.

이후 함수 내부에서 그래프의 정점 개수만큼 서로소 집합을 동적 할당하고 초기화해준다.

이후 모든 정점에서 엣지를 `getAdjacentEdges` 함수를 통해 얻어주고 이를 리스트에 삽입한다.

모든 정점의 엣지를 얻은 후 엣지 리스트를 가중치를 기준으로 오름차순으로 정렬하고 크루스칼 알고리즘을 수행한다.

- `bool Dijkstra(Graph* graph, char option, int vertex)`

그래프에 대한 다익스트라 알고리즘을 수행하는 함수로, 입력 받은 정점과 나머지 모든 정점에 대한 최단 경로와 그 코스트를 계산해 출력한다.

탐색할 정점을 우선순위 큐를 이용해 결정하며 지금까지 탐색한 정점에 대한 최단 경로의 길이를 저장하는 `dist` 배열과 `prev` 배열을 선언하고 초기화한다.

시작 정점에 대한 `dist`는 0, `prev`는 -1로 초기화하며 우선순위 큐에 시작 정점을 삽입하며 다익스트라 알고리즘을 수행한다.

인접 정점에 대하여 현재 정점의 `dist`와 가중치의 합이 해당 정점에서의 `dist`보다 작다면 해당 인접 정점의 `dist`를 현재 정점의 `dist`와 가중치의 합으로 재설정하고 `prev`를 해당 정점으로 설정하고 인접 정점을 우선순위 큐에 삽입한다. 이를 우선순위 큐가 빌 때까지 반복한다.

다익스트라 알고리즘을 모두 수행했으면 `prev`와 `dist`를 이용해 모든 정점에 대한 최단 경로와 그 길이를 출력한다.

- `bool Bellmanford(Graph* graph, char option, int s_vertex, int e_vertex)`

그래프에 대한 벨만포드 알고리즘을 수행하는 함수로, 입력 받은 시작 정점과 도착 정점에 대한 최단 경로를 탐색한다.

다익스트라와 유사하게 `dist`와 `prev` 배열을 선언하고 초기화하여 인접 정점에 대해 현재 `dist`와 가중치의 합이 인접 정점의 `dist`보다 작은지 비교하고, 작다면 인접 정점의 `dist`를 재설정하고 `prev`를 현재 정점으로 설정하는 것을 반복한다. 알고리즘이 종료됐다면 음수 사이클을 확인하고 결과를 출력한다.

- `bool FLOYD(Graph* graph, char option)`

그래프에 대한 플로이드 알고리즘을 수행하는 함수로, 모든 정점 쌍에 대한 최단 경로를 탐색한다.

다익스트라와 유사하게 `dist`와 `prev` 배열을 선언하고 초기화하여 인접 정점에 대해 `dist[i][k]`와 `dist[k][j]`의 합이 `dist[i][j]`보다 작다면 `dist[i][j]`의 값을 재설정하는 것을 모든 정점쌍에 대하여 반복한다.

플로이드 알고리즘을 수행한 후 음수인 사이클을 확인하며 이상이 없다면 결과인 2차원 배열 `dist`를 모두 출력한다.

- Manager

프로그램 전반을 관리하는 클래스이다. 멤버로 로드한 그래프와 파일 출력을 담당하는 `fstream` 객체, `load` 여부를 확인하는 변수를 가진다. Manager 클래스의 멤버 함수로 `GraphMethod`의 함수를 Manager 내부의 Graph에 대해 호출하며, 그래프 파일 parsing 및 그래프 생성 등을 담당한다.

- `void run(const char* command_txt)`

`command.txt`를 읽고 내부의 명령어를 parsing하여 알맞은 manager의 멤버 함수를 실행한다. 에러가 발생하거나 그래프 메소드가 실패하면 알맞은 에러 코드를 출력한다.

- `bool LOAD(const char* filename)`

그래프를 manager 클래스로 불러오는 함수이다. 모든 그래프 메소드는 그래프가 존재하지 않으면 실행할 수 없다. 따라서 그래프 파일에서 그래프 정보를 불러와야 한다.

manager 클래스의 멤버로 graph를 1개 수용할 수 있다. 그래프가 존재하는데 추가로 로드를 실행하는 경우 기존 그래프를 삭제하고 새로운 그래프를 불러온다.

그래프 파일의 첫 줄에는 그래프 유형이 알파벳으로 적혀있고, 그 아래에는 그래프의 정점 개수, 그 아래로는 그래프의 인접 정보가 담겨있다.

행렬형의 경우 그래프의 엣지가 인접 행렬의 형태로 주어진다. 리스트형의 경우 그래프의 엣지가 인접 리스트의 형태로 주어진다.

각 유형별로 parsing하여 그래프에 추가할 수 있도록 구현하였다.

4. Result Screen

```
1    LOAD graph_1.txt
2    PRINT
3    BFS Y 1
4    DFS Y 1
5    KRUSKAL
6    DIJKSTRA Y 1
7    BELLMANFORD Y 1 7
8    FLOYD Y
9    KWANGWOON
```

```
===== LOAD =====
Success
=====
```

```
===== PRINT =====
1-> (2, 6) -> (3, 2)
2-> (4, 5)
3-> (2, 7) -> (5, 3) -> (6, 8)
4-> (7, 3)
5-> (4, 4)
6-> (7, 1)
7-> (5, 10)
=====
```

```
===== PRINT =====
   [1] [2] [3] [4] [5] [6] [7]
[1] 0  6  2  0  0  0  0
[2] 0  0  0  5  0  0  0
[3] 0  7  0  0  3  8  0
[4] 0  0  0  0  0  0  3
[5] 0  0  0  4  0  0  0
[6] 0  0  0  0  0  0  1
[7] 0  0  0  0 10  0  0
=====
```

```
===== BFS =====
Directed Graph BFS result
startvertex: 1
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7
=====
```

```
===== DFS =====
Directed Graph DFS result
startvertex: 1
1 -> 3 -> 6 -> 7 -> 5 -> 4 -> 2
=====
```

```
===== Kruskal =====
[1] 3(2) 2(6)
[2] 4(5)
[3] 5(3) 6(8)
[4] 7(3)
[5]
[6]
[7]
cost: 27
=====
```

```
===== Dijkstra =====
Directed Graph Dijkstra result
[1]x
[2]2 -> 1 (6)
[3]3 -> 1 (2)
[4]4 -> 5 -> 3 -> 1 (9)
[5]5 -> 3 -> 1 (5)
[6]6 -> 3 -> 1 (10)
[7]7 -> 6 -> 3 -> 1 (11)
=====
```

```
===== Bellman-Ford =====
Directed Graph Bellman-Ford result
1 -> 3 -> 6 -> 7
cost: 11
=====
```

```
===== FLOYD =====
Directed Graph FLOYD result
  [1] [2] [3] [4] [5] [6] [7]
[1] 0  6  2  9  5 10 11
[2] 0  0  0  5 18  0  8
[3] 0  7  0  7  3  8  9
[4] 0  0  0 17 13  0  3
[5] 0  0  0  4 17  0  7
[6] 0  0  0 15 11  0  1
[7] 0  0  0 14 10  0 17
=====
```

```
===== KWANGWOON=====
startvertex: 1
1 -> 2 -> 4 -> 7 -> 5
=====
```

5. Consideration

- 고찰

Kruskal을 진행할 때 실질적으로 MST를 생성하진 않고 정렬된 엣지를 출력할 때 실제로 MST를 생성하진 않고 유니온, 파인드 작업을 통해 해당하는 엣지만 표시하도록 구현하였다. 그래프를 반환하거나 그 그래프를 사용하는 조건이 없으므로 실질적으로 Kruskal 함수 내부에서 MST 자체를 담고 있는 리스트는 없었다.

KWANGWOON 알고리즘을 수행할 때 BFS의 작동 방식을 많이 참고하여 구현했다. BFS와 다른 점은 BFS는 인접 정점에 대하여 다수의 정점이 큐에 삽입될 수도 있지만 광운 알고리즘은 오직 하나의 다음 정점을 큐에 삽입하고 수행한다. 벡터를 사용해 정점 정보를 저장하고, 리스트 그래프 구역에서 광운 그래프를 반환할 때 정렬된 상태로 내보내면 추가로 정렬해줄 필요 없이 다음 정점을 선택할 때 벡터의 맨 앞이나 맨 뒤 요소를 선택해 이동하도록 하여 모든 정점의 크기를 비교할 필요 없이 진행이 가능하도록 구현했다.

- 결론

그래프의 경우 개념이 큰 자료형 혹은 데이터구조이기 때문에 C++에선 그래프만을 위한 STL 라이브러리를 찾기 힘들다. 이번 프로젝트를 통해 그래프를 직접 구축하고 이를 사용하는 그래프 탐색 알고리즘을 직접 구현하여 사용해보았다.

또 그래프를 이용해 특정 조건을 만족하는 프로그램을 작성하는 방법도 KWANGWOON 알고리즘을 구현하며 익힐 수 있었다.

데이터구조 수업의 목적은 기초적인 데이터 구조의 원리를 배우고 이를 통해 문제에서 요구하는 형태의 데이터 구조를 직접 구현할 수 있도록 하는 것인데, 이번 프로젝트에선 이를 잘 수행할 수 있었던 것 같다.